

Padrão State aplicado no sistema da FoodDelivery

Autores: João Arthur
Tony Cleriston
Denilson Santos
Vinícius Cerqueira
Gabriel Arlisson

Contexto



- O código inicial não possuía uma estrutura adequada para gerenciar a comunicação entre o cliente e o servidor durante o ciclo de vida de um pedido.
- Tampouco uma forma organizada de administrar os diferentes estados e transições desses pedidos no servidor. Essa ausência de controle tornava o código propenso a se tornar confuso e difícil de manter, especialmente se o comportamento dos pedidos fosse tratado apenas com múltiplas estruturas condicionais (if/else).
- Este padrão fará com que elimine a lógica condicional complexa encapsulando comportamentos específicos de cada estado em classes separadas. Essa abordagem dissemina a lógica condicional e torna o código mais flexível para a incorporação de novos estados.

Prós e Contras



- Esse método assegura que a classe Pedido permaneça enxuta, concentrando-se apenas no gerenciamento geral de estados, enquanto cada classe de estado lida com suas responsabilidades específicas e interage com outros serviços quando necessário.
- Porém, o padrão State pode criar uma complexidade que não precisa, gerar muitas classes para problemas sem muita dificuldade e trazer ligação entre as classes de estado já que elas podem saber sobre as irmãs para cuidar de mudanças.

IOrderState(IPedidoEstado)



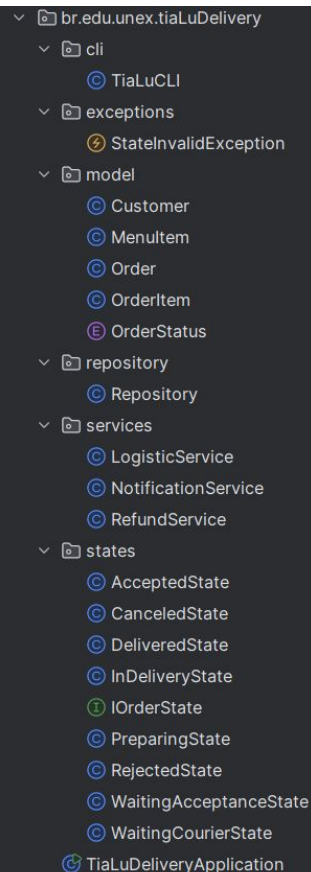
- A interface IPedidoEstado é o coração da nossa implementação do padrão State.
- Diferente da herança de classe tradicional onde herdamos implementações, com interfaces herdamos apenas a assinatura dos métodos.
- Cada um dos nossos 8 estados implementa esta interface, garantindo que todos eles possuam os mesmos métodos

```
package br.edu.unex.tiaLuDelivery.states;

import br.edu.unex.tiaLuDelivery.model.Order;

public interface IOrderState {
    void accept(Order order);
    void reject(Order order, String reason);
    void startPreparation(Order order);
    void finishPreparation(Order order);
    void sendForDelivery(Order order);
    void confirmDelivery(Order order);
    void cancel(Order order);
    String getStatus();
}
```

Estrutura de Pastas



- A organização em pacotes reflete diretamente os princípios de design que aplicamos. Cada pacote tem uma responsabilidade específica e bem definida.
- Começando pelo pacote “states”, que é o núcleo da nossa solução onde cada arquivo é pequeno, focado e fácil de entender.
- O pacote “services” contém as integrações com sistemas externos.
- E o pacote “exceptions” tem nossa exceção customizada StateInvalidException(EstadoInvalidoException).

Mudanças na Classe Pedido

Anteriormente, usando setStatus:

```
public OrderStatus getStatus() {  
    return status;  
}  
  
public void setStatus(OrderStatus status) {  
    this.status = status;  
}
```

Após aplicar o padrão State, a classe Pedido não terá mais a validação de transição e lógica do negócio dentro da classe.

Suas novas responsabilidades são apenas:

- Manter os dados do pedido
- Manter referência para o estado atual
- Delegar ações para o estado atual

```
public void accept() { state.accept( order: this); }  
  
public void reject(String reason) { state.reject( order: this, reason); }  
  
public void startPreparation() { state.startPreparation( order: this); }  
  
public void finishPreparation() { state.finishPreparation( order: this); }  
  
public void sendForDelivery() { state.sendForDelivery( order: this); }  
  
public void confirmDelivery() { state.confirmDelivery( order: this); }  
  
public void cancel() { state.cancel( order: this); }  
  
public String getCurrentStatus() { return state.getStatus(); }
```

AguardandoAceiteEstado



O `WaitingAcceptanceState` (AguardandoAceiteEstado) é o estado inicial. Todo pedido novo começa aqui.

- Caminho 1 – Aceitar: Se o estabelecimento aceita, transita para `AcceptedState` e notifica o cliente.
- Caminho 2 – Rejeitar: Se o estabelecimento rejeita (por exemplo, está fechado), transita para `RejectedState`, salva o motivo da rejeição, e notifica o cliente.
- Caminho 3 – Cancelar: Se o cliente cancela antes do estabelecimento aceitar, transita para `CanceledState`.

```
public class WaitingAcceptanceState implements IOrderState {

    @Override
    public void accept(Order order) {
        System.out.println("Pedido" + order.getId() + " aceito com sucesso");
        order.setState(new AcceptedState());
        NotificationService.notificationCustomer(order.getCustomer(),
            mensagem: "Seu pedido foi aceito");
    }

    @Override
    public void reject(Order order, String reason) {
        System.out.println("Pedido" + order.getId() + " rejeitado, motivo: " + reason);
        order.setState(new RejectedState());
        NotificationService.notificationCustomer(order.getCustomer(),
            mensagem: "Seu pedido foi rejeitado, motivo: " + reason);
    }

    @Override
    public void cancel(Order order) {
        System.out.println("Pedido" + order.getId() + " cancelado pelo cliente");
        order.setState(new CanceledState());
        NotificationService.notificationCustomer(order.getCustomer(),
            mensagem: "Seu pedido foi cancelado");
    }
}
```

AceitoEstado

- O AcceptedState(AceitoEstado) tem apenas duas transições possíveis:
- Transição 1: vai para o Preparo e notifica que o pedido mudou o estado.
- Transição 2: cancelar caso o cliente desista.
- Todas as outras operações são inválidas aqui. Você não pode aceitar novamente um pedido já aceito, não pode rejeitá-lo, e não pode enviá-lo sem preparar primeiro.

```
public class AcceptedState implements IOrderState {  
  
    @Override  
    public void accept(Order order) { throw new StateInvalidException("O pedido já foi aceito"); }  
  
    @Override  
    public void reject(Order order, String reason) { throw new StateInvalidException("O pedido já foi aceito"); }  
  
    @Override  
    public void cancel(Order order) {  
        System.out.println("Pedido" + order.getId() + " cancelado pelo cliente");  
        order.setState(new CanceledState());  
        NotificationService.notificationCustomer(order.getCustomer(),  
            mensagem: "Seu pedido foi cancelado");  
    }  
  
    @Override  
    public void startPreparation(Order order) {  
        System.out.println("Pedido" + order.getId() + " iniciado para preparação");  
        order.setState(new PreparingState());  
        NotificationService.notificationCustomer(order.getCustomer(),  
            mensagem: "Seu pedido foi iniciado para preparação");  
    }  
}
```


PreparandoEstado



- Transição 1: Depois de terminar o preparo do pedido ele passa para WaitingCourierState (AguardandoEntregadorState) onde solicita um entregador via LogisticaService.
- Transição 2: ainda é possível cancelar.

```
@Override
public void cancel(Order order) {
    System.out.println("Pedido" + order.getId() + " cancelado pelo cliente");
    order.setState(new CanceledState());
    NotificationService.notificationCustomer(order.getCustomer(),
        mensagem: "Seu pedido foi cancelado");
}

@Override
public void finishPreparation(Order order) {
    System.out.println("Pedido" + order.getId() + " ficou pronto para entrega");
    order.setState(new WaitingCourierState());
    NotificationService.notificationCustomer(order.getCustomer(),
        mensagem: "Seu pedido ficou pronto para entrega");
}

@Override
public void sendForDelivery(Order order) {
    throw new StateInvalidException("O pedido ainda não teve a preparação finalizada");
}
```

EmEntregaEstado



- O `InDeliveryState(EmEntregaEstado)` é mais restritivo. O pedido saiu para entrega, está com o entregador.
- Aqui, cancelamento não é mais possível. O pedido está a caminho do cliente. Todas as outras operações lançam exceções apropriadas.
- E então de maneira automática e externa como através de webhook ele vai prosseguir de `AguardandoEntregador` para o `EmEntrega` e posteriormente quando confirmado vai para `Entregue`.

Diagrama de Classes

